

Centro Universitário UNA
UC – Modelos, Métodos e Técnicas da Engenharia de Software

Documento de Padrões de Projeto

Nyox Imóveis

Versão do Documento: 1.0



Autores: Carlos Eduardo da Silva
Gustavo Henrique dos Santos
Heitor Zeferino Siqueira
Henrique Oliveira Ferreira
João Vitor Martins Matos
Pedro Henriques Ferreira
Rodrigo Queiroz Vieira Freire

Contagem
22/11/2025

Sumário

1. Objetivo	3
2. Frontend (React)	3
2.1 Arquitetura Baseada em Componentes e Páginas	3
2.2 Gerenciamento de Estado com Hooks	3
2.3 Roteamento Centralizado	3
3. Backend (Node.js/Express)	3
3.1 Singleton	3
3.2 Padrão de Roteador Modular (Modular Router Pattern)	4
3.3 Padrão Middleware	4
4. Padrões Gerais e de Teste	4
4.1 Module Pattern	4
4.2 Behavior-Driven Development (BDD) – Backend	4
4.3 Testes de Componente – Frontend	4
5. Conclusão	5

1. Objetivo

Este documento tem como objetivo detalhar os principais padrões de projeto e as decisões arquiteturais implementadas no projeto, abrangendo o frontend (React), o backend (Node.js/Express) e as estratégias de teste.

2. Frontend (React)

A interface foi construída a partir do React, seguindo padrões que promovem a reutilização e manutenção do sistema.

2.1 Arquitetura Baseada em Componentes e Páginas

- **Padrão:** A aplicação é dividida em Componentes reutilizáveis (elementos de UI como botões, cards, etc.) e Páginas (telas completas que compõem os componentes).
- **Benefício:** Promove a separação de interesses (SoC) e facilita a reutilização de código e simplifica a manutenção.
- **Implementação:** Componentes reutilizáveis ``src/Components/`` e Páginas da aplicação ``src/Pages/``.

2.2 Gerenciamento de Estado com Hooks

- **Padrão:** O estado dos componentes é gerenciado localmente através do React Hooks.
- **Benefício:** Simplifica a lógica de estado dentro dos componentes funcionais, tornando-os mais legíveis e fáceis de testar.
- **Implementação:** Uso de ``useState`` e ``useEffect`` para controlar o ciclo de vida e os dados dos componentes. Ex.: ``src/Pages/PainelAdm/PainelAdm.jsx`` utiliza hooks para buscar e exibir a lista de imóveis.

2.3 Roteamento Centralizado

- **Padrão:** Um único arquivo de configuração define todas as rotas da aplicação, mapeando URLs para os componentes de página correspondentes.
- **Benefício:** Oferece um local único para entender e gerenciar a navegação da aplicação
- **Implementação:** O arquivo ``src/Pages/Router/App.jsx`` utiliza a biblioteca ``react-router-dom`` para definir as rotas.

3. Backend (Node.js/Express)

O servidor da API foi construído com Node.js e Express, adotando padrões que garantem escalabilidade e organização.

3.1 Singleton

- **Padrão:** Garante que uma única instância do cliente de banco de dados (Supabase) seja criada e compartilhada por toda a aplicação.
- **Benefício:** Melhora a performance ao evitar a sobrecarga de criar uma nova conexão a cada requisição.
- **Implementação:** O arquivo ``src/api/src/Connection/Supabase.js`` cria e exporta uma única instância do cliente, que é importada nos módulos de rota quando necessário.

3.2 Padrão de Roteador Modular (Modular Router Pattern)

- **Padrão:** As rotas da API são organizadas em módulos separados com base no recurso que manipulam (ex: ``Imoveis``, ``User``).
- **Benefício:** Mantém o arquivo principal do servidor (``index.js``) limpo e desacoplado da lógica de roteamento, facilitando a manutenção.
- **Implementação:** O arquivo ``src/api/src/index.js`` importa e delega as rotas para os arquivos em ``src/api/src/Routes/``.

3.3 Padrão Middleware

- **Padrão:** Funções que interceptam o ciclo de requisição-resposta para executar tarefas transversais, como autenticação, logging, parsing de dados e tratamento de erros.
- **Benefício:** Encapsula lógicas comuns, evitando a repetição de código nos manipuladores de rota.
- **Implementação:** Uso de ``cors`` e ``express.json()`` em ``src/api/src/index.js`` e uso do ``multer`` como middleware em ``src/api/src/Routes/Imoveis.js`` para processar uploads de arquivos.

4. Padrões Gerais e de Teste

4.1 Module Pattern

- **Padrão:** Utiliza o sistema de módulos nativo do JavaScript (ESM) e Node.js (CommonJS) para encapsular código em arquivos, expondo apenas uma interface pública;
- **Benefício:** Fundamental para a organização, encapsulamento e reutilização de código em todo o projeto.
- **Implementação:** Presente em todos os arquivos ``js`` e ``jsx`` do projeto.

4.2 Behavior-Driven Development (BDD) – Backend

- **Padrão:** Os testes da API são escritos em linguagem natural (Gherkin), descrevendo o comportamento do sistema sob a perspectiva do usuário.;
- **Benefício:** Cria uma documentação viva e garante que a API se comporte conforme o esperado.
- **Implementação:** Arquivos ``feature`` em ``src/api/tests/features/`` (ex: ``imoveis.feature``) e Arquivos de steps em ``src/api/tests/steps/`` que implementam os cenários.

4.3 Testes de Componente – Frontend

- **Padrão:** Cada componente React é renderizado e testado de forma isolada para verificar sua aparência e comportamento em resposta a interações;
- **Benefício:** Garante a confiabilidade dos blocos de construção da UI.
- **Implementação:** Arquivos `*.test.jsx` no diretório `tests/` (ex: `tests/Components/AnuncioCard.test.jsx`).

5. Conclusão

Os padrões de projeto e práticas arquiteturais apresentados desempenham um papel essencial na construção de sistemas modernos, escaláveis e de fácil manutenção. Ao aplicar estratégias de modularização, separação de responsabilidades, reutilização e testes bem definidos, o desenvolvedor garante não apenas um código mais limpo, mas também um produto final mais confiável e eficiente.

A combinação entre um frontend organizado em componentes, um backend estruturado em padrões como Singleton, Middleware e Router Pattern, além de metodologias de teste como BDD e testes de componentes, cria uma base sólida para qualquer aplicação. Esses conceitos não só otimizam o processo de desenvolvimento, como também melhoram a experiência do usuário e deixa o sistema mais completo.